

IoT Challenge #3 – Node-Red & LoRaWAN

Challenge Report

Anasuya Satapathy
Golsa Shokri

May 2026

Contents

1	Introduction	3
2	Node-RED flow description	3
3	Node-by-Node Explanation	3
3.1	Initialization branch	3
3.1.1	AUTO: Load CSV on deploy (inject)	3
3.1.2	Read challenge3.csv (file in)	4
3.1.3	Parse CSV and store by Packet Number (function)	4
3.1.4	AUTO: Write id_log.csv header and AUTO: Write filtered_elems.csv header	4
3.2	Publisher branch	4
3.2.1	Every 1 second (inject)	4
3.2.2	Generate random ID + timestamp (function)	4
3.2.3	Publish challenge3/id_generator (mqtt out)	4
3.2.4	Append row to id_log.csv (function) + APPEND to id_log.csv (file) .	5
3.3	Subscriber / processing branch	5
3.3.1	Subscribe challenge3/id_generator (mqtt in)	5
3.3.2	Count messages (function with 2 output branch)	5
3.3.3	Compute N (function)	5
3.3.4	Lookup packet N in CSV (function)	5
3.3.5	Classify: ZBEE_ZCL / Link Status / Ignore (function)	5
3.3.6	Route by packet type (switch)	5
3.4	ZigBee processing path	6
3.4.1	Build ZigBee MQTT payload (function)	6
3.4.2	Rate limit: 10 msg / minute (delay)	6
3.4.3	Publish ZigBee message (mqtt out)	6
3.4.4	Check RMS Current + Voltage + Active Power (function, 3 outputs) .	6
3.4.5	Append rows to filtered_elems.csv (function) + APPEND to filtered_elems.csv (file)	6
3.4.6	RMS Voltage Chart and RMS Current Chart (ui_chart)	7
3.5	Link Status processing path	7
3.5.1	Process Link Status - update cost map (function)	7
3.6	Finalization branch	7
3.6.1	Save outgoing_cost.csv & build ThingSpeak sequence (function, 2 outputs)	7
3.6.2	OVERWRITE outgoing_cost.csv (file)	7

3.6.3	Split into individual ThingSpeak msg (function)	7
3.6.4	Rate limit: 1 msg / 20 seconds (delay)	8
3.6.5	GET ThingSpeak update (http request)	8
3.6.6	ThingSpeak response (debug)	8
4	Generated CSV Files	8
4.1	id_log.csv (excerpt)	8
4.2	filtered_elems.csv (excerpt)	9
4.3	outgoing_cost.csv (excerpt)	9
5	ThingSpeak Channel	9
6	Node-Red Dashboard Charts	11
7	Summary	12

1 Introduction

This report describes the Node-Red flow developed for IoT Challenge 3. The challenge requires processing a ZigBee capture file (`challenge3.csv`) in real time through MQTT messaging, filtering specific packet types, and forwarding relevant data to ThingSpeak. The flow runs locally on a Mosquitto broker (`localhost:1884`) and produces three CSV files (`id_log.csv`, `filtered_elems.csv`, `outgoing_cost.csv`) plus the ThingSpeak channel updates required by the assignment.

2 Node-RED flow description

The flow is logically organized into four cooperating branches, all running on the same tab

1. **Initialization branch** – loads `challenge3.csv` into memory and writes the headers of the two output CSV files written in *append* mode.
2. **Publisher branch** – generates random IDs every second and publishes them on the local Mosquitto broker.
3. **Subscriber / processing branch** – receives the IDs, computes $N = \text{ID} \bmod 5218$, looks up the corresponding packet, and routes it to either the ZigBee processing path or the Link Status processing path.
4. **Finalization branch** – after exactly 200 received IDs, builds `outgoing_cost.csv` and pushes the data of the smallest source address to ThingSpeak.

A picture of the entire flow is reported in Figure below.

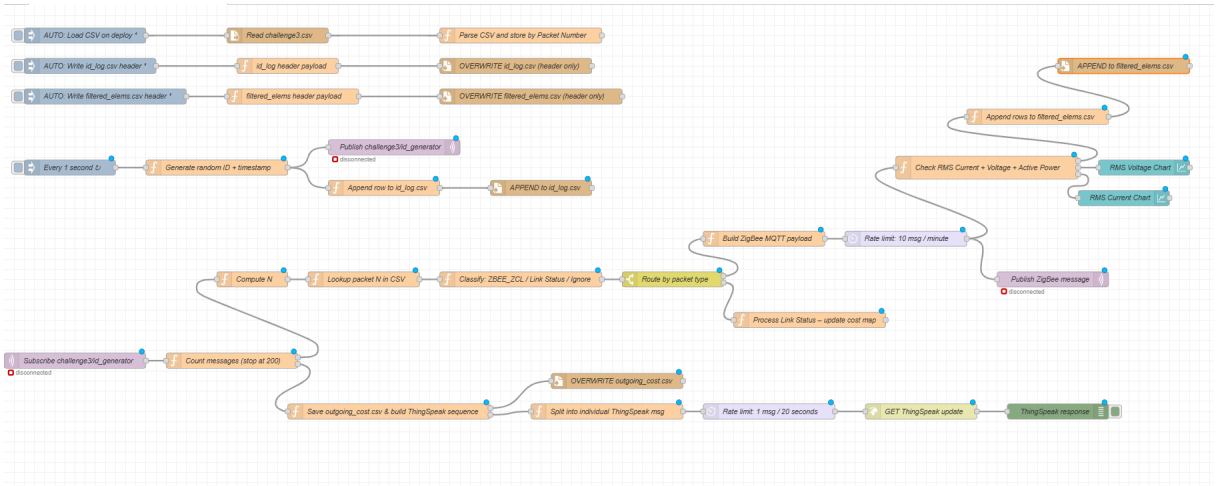


Figure 1: node flow

3 Node-by-Node Explanation

3.1 Initialization branch

3.1.1 AUTO: Load CSV on deploy (inject)

Inject node configured with *Inject once after 0.5 s* so that, every time the flow is deployed, `challenge3.csv` is automatically reloaded into the flow context, without requiring any manual click.

3.1.2 Read challenge3.csv (file in)

Reads the entire content of `challenge3.csv` as a UTF-8 string and forwards it as `msg.payload` to the next node.

3.1.3 Parse CSV and store by Packet Number (function)

Parses the CSV string into rows using a small custom parser that handles double-quoted fields containing commas (which appear in the Command String column. Each parsed row is stored in a JavaScript object keyed by its `Packet Number`, so that later lookups by N are direct and do not require scanning the full dataset ($O(1)$ access) . The resulting object is saved in `flow.csvData`; the node status displays “Loaded 5218 packets” once parsing is complete.

3.1.4 AUTO: Write id_log.csv header and AUTO: Write filtered_elems.csv header

Two parallel mini-pipelines, each made of an inject, a function and a file node. They are triggered automatically on deploy and they overwrite `id_log.csv` and `filtered_elems.csv` with their header line only:

No.,ID,TIMESTAMP

No.,Timestamp,Sequence Number,Attribute,Status,Data Type,Data Value

The two header function nodes also reset the relevant flow variables (counters, stop flag, the ordered cost structures used by the Link Status branch) so that every deploy starts from a clean state.

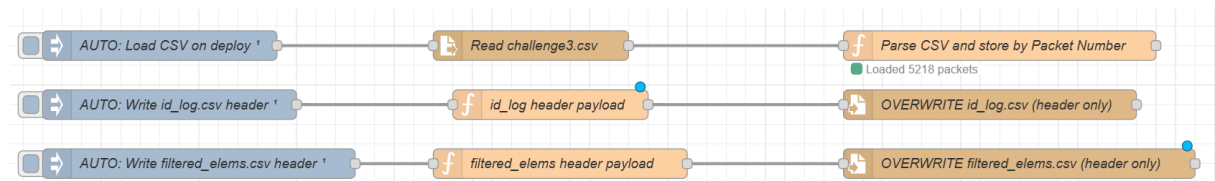


Figure 2: Initialization branch

3.2 Publisher branch

3.2.1 Every 1 second (inject)

Inject node configured with repeat every 1 second, providing the periodic trigger.

3.2.2 Generate random ID + timestamp (function)

Generates an integer in $[0, 30000]$ via `Math.floor(Math.random() * 30001)` and the current UNIX timestamp in seconds. It builds the JSON payload `{"id": <id>, "timestamp": <ts>}` and stores the two values also in `msg._id` and `msg._ts` so that the downstream CSV writer can log them. If the flag `flow.stop_generation` is set (which happens once 200 messages have been processed by the subscriber), the node returns `null` and updates its status to Stopped, so that the publisher does not keep flooding the broker after the run is complete.

3.2.3 Publish challenge3/id_generator (mqtt out)

Publishes the JSON payload to the topic `challenge3/id_generator` on the local broker (`localhost:1884`, QoS 0).

3.2.4 Append row to id_log.csv (function) + APPEND to id_log.csv (file)

The function maintains an incremental row counter in `flow.id_counter` and produces a CSV line of the form `No.,ID,TIMESTAMP`. The file node appends the line in append mode, without adding extra newlines.

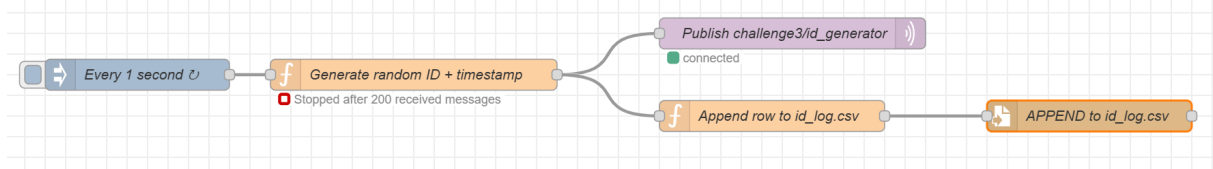


Figure 3: Publisher branch

3.3 Subscriber / processing branch

3.3.1 Subscribe challenge3/id_generator (mqtt in)

Subscribes to the same topic on the local broker. The first node downstream is the counter gate, so all received messages are immediately counted – even if they will later be ignored, in line with the rule “*ignored messages still count toward the 200-message limit*”.

3.3.2 Count messages (function with 2 output branch)

Implements the global stop condition. On every incoming message it increments `flow.received_count`. If the counter exceeds 200 the message is silently dropped and `flow.stop_generation` is set so that the publisher stops as well. On the **200th** message the function does two things at the same time: it forwards the message on output 1 (so that the 200 th message is still processed normally), and it releases a `finalize_after_200` trigger on output 2 that kicks off the writing of `outgoing_cost.csv` and the ThingSpeak sequence. The trigger is generated before parsing the JSON payload, so even a malformed 200 th message would not prevent the finalization from running.

3.3.3 Compute N (function)

It simply computes $\text{msg}._N = \text{msg}._\text{sub_id} \% 5218$.

3.3.4 Lookup packet N in CSV (function)

Retrieves the row whose `Packet Number` equals `N` from `flow.csvData` and attaches it as `msg._row`. If `N` is not present in the dataset, it releases a warning and drops the message.

3.3.5 Classify: ZBEE_ZCL / Link Status / Ignore (function)

Inspects the row and tags it with one of three logical types, written into `msg._type`:

- `zbee_zcl` if the *Command String* contains the substring `Layer ZBEE_ZCL`;
- `link_status` if the *Packet Type* field contains `Link Status`;
- `ignore` otherwise.

3.3.6 Route by packet type (switch)

Standard `switch` node that routes the message based on `msg._type`: output 1 goes to the ZigBee path, output 2 to the Link Status path. The `ignore` case is not wired and is silently dropped, while still having contributed to the 200-message count thanks to the upstream gate.

3.4 ZigBee processing path

3.4.1 Build ZigBee MQTT payload (function)

Builds the JSON payload required, copying the `Source Address`, `Destination Address`, `Source/Destination Address ZigBee` and `Command String` fields from the CSV row. The MQTT topic is set dynamically to `ZigBee/<Device Name ZigBee Source>` (e.g. `ZigBee/Coordinator`), matching the topic field embedded in the payload itself, as clarified in the forum.

In addition, the function parses the Python-style `Command String` into a JavaScript object (`msg._cmd`) by replacing single quotes with double quotes and the `True/False/None` keywords with `true/false/null`. This parsed object is consumed downstream by the power-attribute extractor.

3.4.2 Rate limit: 10 msg / minute (delay)

Configured in rate mode, 10 messages every 1 minute, with `drop = false` so that excess messages are queued rather than dropped. The output of this node is fanned out to both the MQTT publisher and the power-attribute extractor, so the filtering happens after the message has been published, as required.

3.4.3 Publish ZigBee message (mqtt out)

Publishes the rate-limited message to the local broker. The topic field of the node is left empty, so it picks up the dynamic `msg.topic` that was set upstream.

3.4.4 Check RMS Current + Voltage + Active Power (function, 3 outputs)

This is the most articulated function in the flow. It walks through the parsed *Command String* and extracts only the attributes that match *RMS Current*, *RMS Voltage* or *Active Power*. The logic is the following:

1. **Hard filter on the packet:** if any of the three lists `Attribute`, `Status` or `Data Type` is empty (or absent), the packet is discarded. This is what excludes *Read Attributes* requests (which contain no status and no data type), *Report Attributes* (which have data type but no status) and *Default Response* (which has no array of attributes), keeping only *Read Attributes Response* packets, which are the only ones that carry all the required fields.
2. **Per-type value pointers:** `Int16` and `Uint16` values appear in the *Command String* as two separate lists. The function maintains one cursor per data type and advances it for *every* attribute (target or not), so the position-based matching between attribute, status, data type and value is preserved.
3. **Per-attribute completeness check:** for each candidate attribute, it verifies that its corresponding status, data type and value all exist; otherwise the attribute is skipped.
4. **Output split:** the rows that survive the filtering are forwarded as a single message on output 1 (towards the CSV appender). In addition, RMS Voltage and RMS Current values are emitted on outputs 2 and 3 respectively, feeding the two charts.

3.4.5 Append rows to filtered_elems.csv (function) + APPEND to filtered_elems.csv (file)

The function consumes the array of rows produced upstream, increments the incremental row counter `flow.filtered_counter` and builds the CSV text in the format `No.,Timestamp,Sequence`

Number,Attribute,Status,Data Type,Data Value. Fields containing commas, quotes or new-lines are properly escaped. The file node appends in append mode.

3.4.6 RMS Voltage Chart and RMS Current Chart (**ui_chart**)

Two Node-Red Dashboard chart widgets configured as line charts. They use the message arrival time as the x-axis (default behaviour) and display the numeric value as the y-axis.

3.5 Link Status processing path

3.5.1 Process Link Status - update cost map (**function**)

Parses the Command String of a Link Status packet and extracts the **Links** array. For each (Address, Outgoing Cost) pair, the function updates an ordered data structure made of:

- an array `outgoing_cost_rows` containing `{src, dst, cost}` entries in the order in which they first appeared, and
- a hash `outgoing_cost_idx` mapping "`src|dst`" to the position of the entry in the array.

When a later Link Status packet updates an already-known link, only the cost field of the existing entry is overwritten; its position – and therefore its row number in the final CSV – is preserved. This matches the indication that `outgoing_cost.csv` must keep the arrival order of the Link Status packets, and that the row number must remain attached to the original entry when the link is updated.

3.6 Finalization branch

3.6.1 Save `outgoing_cost.csv` & build ThingSpeak sequence (**function, 2 outputs**)

Triggered exactly once, when the counter gate emits the `finalize_after_200` message. Two things happen here:

1. The CSV `outgoing_cost.csv` is written by iterating over `outgoing_cost_rows` *in arrival order*, with no sorting. The output goes to the file node `OVERWRITE outgoing_cost.csv`.
2. The ThingSpeak sequence is built: the function selects the smallest source address (treating addresses as hexadecimal numbers), filters the rows belonging to it and sorts them by destination address in ascending hexadecimal order. The resulting list of `{dst, cost}` entries is forwarded to the splitter on output 2.

The reordering is therefore confined to the ThingSpeak path and does not affect the CSV.

3.6.2 `OVERWRITE outgoing_cost.csv` (**file**)

File node in overwrite mode that receives the final CSV string and writes it to disk in one shot.

3.6.3 Split into individual ThingSpeak msg (**function**)

Receives the ordered list of `{dst, cost}` pairs and emits one message per pair via `node.send(...)`. Each emitted message carries the cost in `msg.payload`, the destination address in a custom `msg._dst` property (for logging) and, most importantly, the pre-built ThingSpeak update URL in `msg.url` (`https://api.thingspeak.com/update?api_key=[...]&field1=<cost>`), ready to be consumed by the downstream HTTP request node.

3.6.4 Rate limit: 1 msg / 20 seconds (delay)

Configured in rate mode, 1 message every 20 seconds, with queueing enabled. This enforces the 20-second-per-update rate limit that we have to respect on ThingSpeak’s free tier.

3.6.5 GET ThingSpeak update (http request)

Performs an HTTP GET on the URL set in the message. The `paytoqs` option is set to ignore because the URL already contains the required query string.

3.6.6 ThingSpeak response (debug)

Debug node attached to the HTTP response, used during development to verify that ThingSpeak was accepting the updates (it returns the entry ID on success, or 0 on rate-limit violations).

4 Generated CSV Files

The flow produced three CSV files. For each one we report below an extract of the first rows together with the global statistics of the file; the full content is included in the delivery zip and not reproduced here in order to keep the report compact. A summary of the global counts is given in Table 1.

Table 1: Summary of the produced CSV files.

File	Data rows	Notes
<code>id_log.csv</code>	200	one per generated ID, publisher self-stops at 200
<code>filtered_elems.csv</code>	24	8 <i>Read Attributes Response</i> packets $\times$ 3 attrs
<code>outgoing_cost.csv</code>	70	links from 9 distinct ZigBee source addresses

4.1 `id_log.csv` (excerpt)

The first ten rows are reported in Table 2. The IDs span the entire allowed range $[0, 30000]$ and the `TIMESTAMP` field grows on average by 1 s per row, with occasional small gaps caused by the OS scheduling of the inject node.

Table 2: First 10 rows of `id_log.csv`.

No.	ID	TIMESTAMP
1	13014	1778513756
2	3303	1778513757
3	6301	1778513758
4	26742	1778513759
5	27725	1778513760
6	24373	1778513761
7	27144	1778513762
8	29752	1778513763
9	19905	1778513764
10	22055	1778513765

4.2 filtered_elems.csv (excerpt)

This file contains the rows extracted from ZBEE_ZCL packets whose *Command String* includes *Active Power*, *RMS Current* or *RMS Voltage* attributes carrying actual response data (i.e. *Read Attributes Response* packets). The rows always appear in groups of three sharing the same timestamp and sequence number, since each qualifying packet contributes the three target attributes together. The first 12 rows are shown in Table 3.

Table 3: First 12 rows of `filtered_elems.csv`.

No.	Timestamp	Seq	Attribute	Status	Data Type	Value
1	1778513758	139	Active Power	Success	16-Bit Signed Integer	0
2	1778513758	139	RMS Current	Success	16-Bit Unsigned Integer	0
3	1778513758	139	RMS Voltage	Success	16-Bit Unsigned Integer	221
4	1778513778	135	Active Power	Success	16-Bit Signed Integer	0
5	1778513778	135	RMS Current	Success	16-Bit Unsigned Integer	0
6	1778513778	135	RMS Voltage	Success	16-Bit Unsigned Integer	222
7	1778513838	110	Active Power	Success	16-Bit Signed Integer	0
8	1778513838	110	RMS Current	Success	16-Bit Unsigned Integer	0
9	1778513838	110	RMS Voltage	Success	16-Bit Unsigned Integer	218
10	1778513859	9	Active Power	Success	16-Bit Signed Integer	0
11	1778513859	9	RMS Current	Success	16-Bit Unsigned Integer	0
12	1778513859	9	RMS Voltage	Success	16-Bit Unsigned Integer	222

4.3 outgoing_cost.csv (excerpt)

This file contains one row per (source, destination) ZigBee address pair encountered in Link Status packets, with the most recent outgoing cost observed. As required, the rows appear in the order in which the corresponding link was first seen in the trace; subsequent updates only modify the cost value, leaving the row number untouched. The 9 distinct source addresses observed are 0x0000, 0x0112, 0x1cd8, 0x1e15, 0x3d95, 0x4e11, 0xa706, 0xe1a6 and 0xec7f.

In this particular run the first Link Status packet that was processed came from 0xec7f, so its links occupy the first 8 rows of the CSV. The Coordinator (0x0000) was the third source observed and its 8 links occupy rows No. 24–31, in the middle of the CSV. It is worth noting that the smallest source by hexadecimal value is still 0x0000, so even though it does not appear at the top of the CSV, it is still the source whose costs are sent to ThingSpeak. This confirms that the arrival-order CSV and the hex-sorted ThingSpeak sequence are computed independently, as required by the assignment. The first 14 rows are shown in Table 4.

5 ThingSpeak Channel

The smallest source address (treating hex values numerically) among all sources in `outgoing_cost.csv` is 0x0000, i.e. the ZigBee Coordinator. Its destination addresses were sorted in ascending hexadecimal order and the corresponding outgoing costs were sent to ThingSpeak as `field1`, one message every 20 seconds. The sequence is reported in Table 5.

The ThingSpeak channel is public.

- Channel ID: **3340710**
- Channel name: `Iot_challenge_3`
- Channel URL: <https://thingspeak.com/channels/3340710>

All 8 data points were successfully sent (entry IDs 1–8, one every 20 seconds) using the HTTP GET API:

Table 4: First 14 rows of `outgoing_cost.csv`.

No.	Source	Destination	Cost
1	0xec7f	0x0000	1
2	0xec7f	0x0112	7
3	0xec7f	0x1cd8	1
4	0xec7f	0x1e15	1
5	0xec7f	0x3d95	1
6	0xec7f	0x4e11	1
7	0xec7f	0xa706	3
8	0xec7f	0xe1a6	1
9	0x4e11	0x0000	1
10	0x4e11	0x0112	7
11	0x4e11	0x1cd8	1
12	0x4e11	0x1e15	3
13	0x4e11	0x3d95	2
14	0x4e11	0xa706	0

Table 5: ThingSpeak data sent (source = 0x0000).

Order sent	Destination	Outgoing Cost (field1)
1	0x0112	7
2	0x1cd8	1
3	0x1e15	3
4	0x3d95	3
5	0x4e11	3
6	0xa706	5
7	0xe1a6	1
8	0xec7f	1

`https://api.thingspeak.com/update?api_key=<our-write-key>&field1=<cost>`

The actual write key is hard-coded in the function node `Split into individual ThingSpeak msg`; we have replaced it with the placeholder `<our-write-key>` in this report for security reasons, since the API key would allow anyone reading this PDF to write data on our channel.

The Field 1 chart on the channel is shown in Figure below.

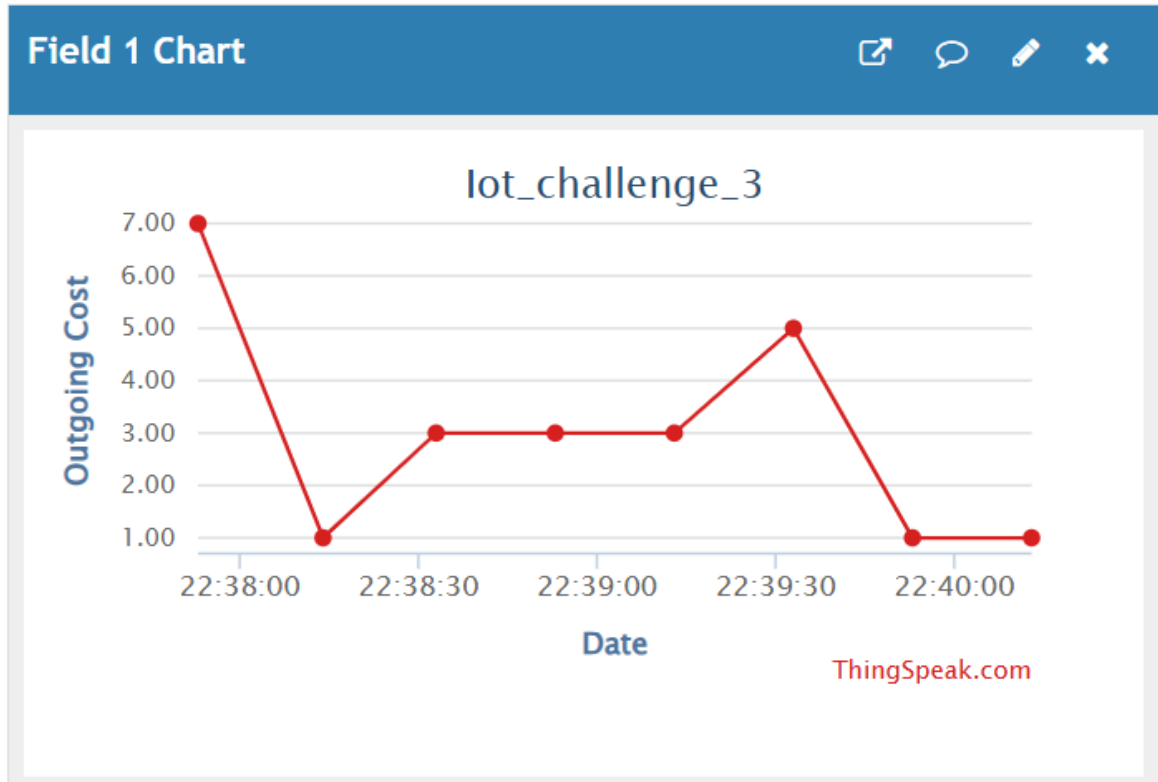


Figure 4: ThingSpeak Field 1 chart with all 8 outgoing-cost values sent in destination-address order (channel [3340710](#)).

6 Node-Red Dashboard Charts

The Node-Red dashboard displays two real-time line charts that get updated every time a ZBEE_ZCL message carrying RMS attributes is processed (Figure 5).

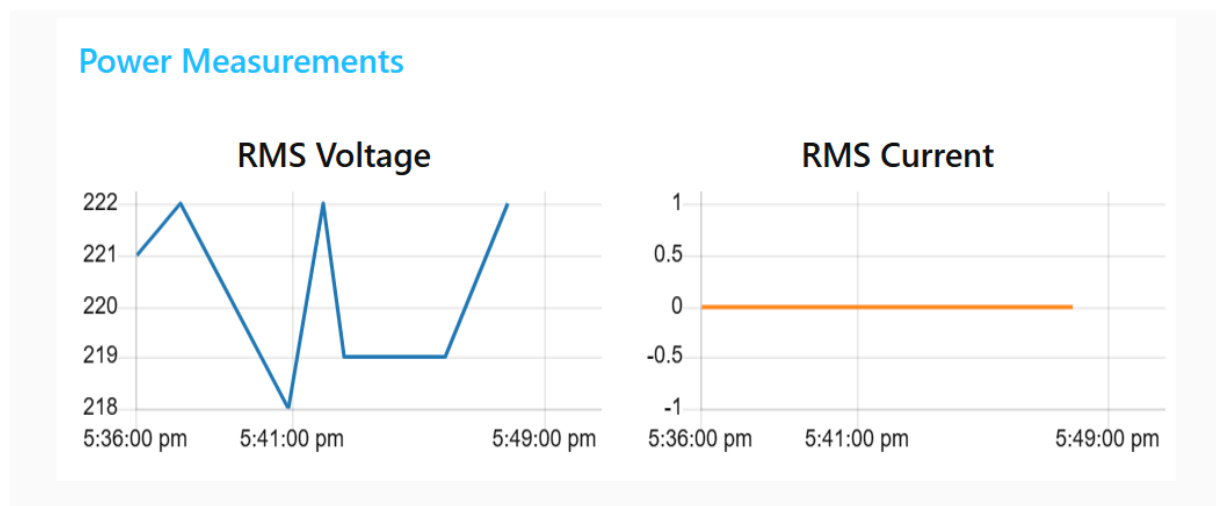


Figure 5: Node-Red Dashboard – RMS Voltage (left) and RMS Current (right) over time, populated from `filtered_elems.csv` data.

The RMS Voltage readings range between 218 and 222 V, which is fully consistent with the European mains supply (≈ 220 V RMS). The RMS Current trace is constantly equal to 0,

suggesting that the monitored device was in standby mode during the captured trace (no actual load was drawing current).

7 Summary

The complete Node-Red flow can be summarized as follows:

- **Publisher branch:** Inject → Generate ID → MQTT Out + CSV log (1 msg/sec).
- **Subscriber branch:** MQTT In → Counter (limit 200) → Compute N → Lookup CSV → Switch.
 - *ZBEE_ZCL path:* Build payload → Rate limiter (10/min) → MQTT publish → attribute filter → `filtered_elems.csv` + dashboard charts.
 - *Link Status path:* ordered cost-map update → (at the 200th message) CSV write → ThingSpeak HTTP GET (1 msg/20s).
 - *Ignore path:* dropped, but counted toward the 200-message limit.

All assignment requirements have been verified against the produced output files and the public ThingSpeak channel.

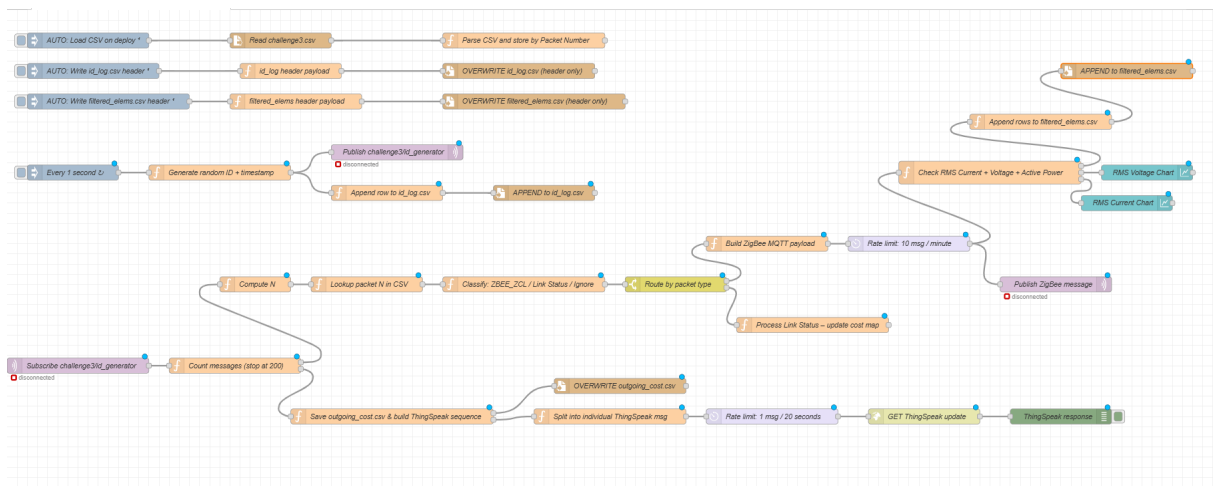


Figure 6: The complete node flow chart

